

Тендер

CUDA

Compute Unified Device Architecture

Параллельные вычисления на GPU NVIDIA

Разбор технологии, развёртывание, демонстрация на CuPy и PyTorch

Содержание

1. Что такое CUDA и зачем она нужна	2
1.1. Определение	2
1.2. Зачем это нужно	2
1.3. Типовые сценарии применения	2
1.4. Как устроено внутри (кратко)	3
2. Развёртывание CUDA на компьютере (Windows)	5
2.1. Шаг 1. Проверка совместимости видеокарты	5
2.2. Шаг 2. Установка драйвера	5
2.3. Шаг 3. CUDA Toolkit (опционально)	5
2.4. Шаг 4. cuDNN	5
2.5. Шаг 5. Установка Python-стека	6
2.6. Шаг 6. Проверка работоспособности	6
2.7. Замечания	6
3. Демонстрация: CuPy против NumPy	7
3.1. Проверка окружения	7
3.2. Корректное измерение времени GPU	7
3.3. Бенчмарк 1. Матричное умножение	7
3.4. Бенчмарк 2. Поэлементные операции	8
3.5. Бенчмарк 3. БПФ (FFT)	8
3.6. Визуализация результатов	8
4. Установка cuDNN + PyTorch и демонстрация CUDA в PyTorch	9
4.1. Что даёт cuDNN	9
4.2. Установка PyTorch (Windows, нативно, без контейнеров)	9
4.3. Бенчмарк PyTorch: CPU против CUDA	9
4.4. Свёртка через cuDNN	11
4.5. Полный цикл обучения	11
4.6. Когда GPU не быстрее	12
5. Заключение	13
6. Приложение. Установка под Linux	14

Что такое CUDA и зачем она нужна

1.1. Определение

CUDA — это технология компании **NVIDIA**, позволяющая использовать видеокарту как универсальный вычислитель, а не только как ускоритель графики. Полное название — **Compute Unified Device Architecture**.

CUDA состоит из трёх частей:

- **аппаратная** — потоковые мультипроцессоры (Streaming Multiprocessor, SM), CUDA-ядра, тензорные ядра (Tensor Core), иерархия памяти GPU;
- **программная** — компилятор `nvcc`, среда исполнения CUDA Runtime, драйвер, отладчик и профилировщик;
- **библиотеки и интеграции** — `cuBLAS`, `cuDNN`, `cuFFT`, `cuRAND`, `Thrust`, `NCCL`, `TensorRT`, а также биндинги для Python (CuPy, Numba, PyTorch, TensorFlow, JAX), C/C++, Fortran и Julia.

1.2. Зачем это нужно

Чтобы понять смысл CUDA, сравним два типа процессоров.

CPU содержит небольшое число мощных ядер (обычно 8–24). Каждое ядро универсально и оптимизировано под быстрое выполнение одной последовательности команд. Это эффективно, когда задача требует сложной логики, ветвлений и работы с разнородными данными.

GPU содержит тысячи простых ядер, рассчитанных на одновременное выполнение одной и той же операции над разными элементами данных. Каждое ядро по отдельности слабее ядра CPU, но за счёт массовой параллельности GPU обрабатывает большие массивы данных в десятки и сотни раз быстрее.

Таким образом, CPU выигрывает на задачах с малым объёмом данных и сложным управлением, а GPU — на задачах **массового параллелизма по данным**: матричные операции, преобразования сигналов, обучение нейронных сетей, физические и финансовые симуляции.

CUDA предоставляет программную модель, компилятор и библиотеки, с помощью которых разработчик описывает вычисление один раз, а среда исполнения автоматически распределяет его по тысячам параллельных потоков GPU.

Количественную разницу между двумя архитектурами иллюстрирует следующая таблица:

Характеристика	CPU (Intel Core Ultra 7 155H)	GPU (RTX 4060 Laptop)
Ядер	16 (6P + 8E + 2 LPE)	3072 CUDA + 96 Tensor + 24 RT
Потоков (hardware threads)	22	десятки тысяч одновременно
Тактовая частота (boost)	≈ 4.8 ГГц (P-ядра)	≈ 2.4 ГГц
FP32 пиковая	≈ 0.5 TFLOPS	≈ 15 TFLOPS
Память	LPDDR5x-7467, 16 ГБ, ≈ 120 ГБ/с	GDDR6, 8 ГБ, 272 ГБ/с

1.3. Типовые сценарии применения

1. **Глубокое обучение**: обучение и инференс нейронных сетей (PyTorch, TensorFlow, JAX).
2. **Линейная алгебра большой размерности**: `cuBLAS`, `cuSOLVER`.

3. **Научные вычисления:** молекулярная динамика, гидродинамика, моделирование Монте-Карло.
4. **Обработка сигналов и изображений:** cuFFT, OpenCV CUDA, NPP.
5. **Финансовое моделирование, криптография, рендеринг, симуляции физики в играх.**

1.4. Как устроено внутри (кратко)

Внутри CUDA две связанные иерархии: как задача **разрезается на параллельные кусочки** и где **лежат данные**, к которым эти кусочки обращаются. От обеих напрямую зависит, насколько быстро в итоге считает GPU.

Иерархия потоков. Задача разбивается на много мелких «исполнителей» — потоков. Каждый поток обычно обрабатывает один элемент данных и по своему индексу понимает, какой именно. Потоки складываются вверх по уровням:

Уровень	Что это и зачем нужен
Поток (thread)	Мельчайшая единица параллелизма. Имеет свои регистры и индекс в задаче.
Блок (block, до 1024 потоков)	Работает на одном мультипроцессоре (SM). Внутри блока потоки умеют обмениваться данными через быструю общую память и синхронизироваться.
Сетка (grid)	Все блоки одной задачи. Запускается с CPU одной командой и целиком выполняет одно «ядро» (kernel — функция, которая крутится на GPU).

Физически потоки исполняются группами по 32 — **варп** (warp). Все 32 потока варпа выполняют одну и ту же инструкцию одновременно. Если внутри кода встречается `if/else` и часть потоков уходит в одну ветку, часть — в другую, GPU вынужден исполнять обе ветки последовательно. Это называется **дивергенцией варпа** и заметно бьёт по скорости, поэтому в CUDA по возможности избегают сильно ветвящегося кода.

Иерархия памяти. Память GPU расслоена по скорости и объёму:

Уровень	Кому видна	Особенности
Регистры	Поток	Самая быстрая, но очень мало (десятки штук на поток).
Общая память (shared)	Блок	Десятки–сотни КБ на мультипроцессор. Используется для обмена данными внутри блока. На порядок быстрее глобальной.
Глобальная память (VRAM)	Вся сетка + CPU через копирование	Большая (8–24 ГБ на десктопных RTX), но в сотни раз медленнее регистров.
Кэши L1 / L2	Аппаратные	Прозрачные для программы; ускоряют повторные обращения к глобальной памяти.

Главное правило производительности: чем чаще поток ходит в глобальную память и чем меньше использует регистры и общую память, тем медленнее работает. Поэтому грамотно написанное CUDA-

ядро может быть в разы быстрее «наивного» — и наоборот, наивное копирование «всё в global» убивает любую скорость.

Что из этого нужно знать пользователю Python. При работе через CuPy и PyTorch всю описанную иерархию за вас разруливают готовые библиотеки (`cuBLAS` , `cuDNN` и др.) — достаточно создавать тензоры на устройстве `cuda` и вызывать обычные операции. Но если профилировщик показывает простой GPU, или если вы пишете своё ядро через `cupy.RawKernel` или `numba.cuda.jit` , знание потоки/блоки/память пригодится напрямую.

Развёртывание CUDA на компьютере (Windows)

В этой главе пошаговая инструкция установки CUDA Toolkit и cuDNN на Windows 10/11 для машины с видеокартой **NVIDIA RTX**. Аналогичные шаги для Linux описаны в приложении.

2.1. Шаг 1. Проверка совместимости видеокарты

Откройте PowerShell и выполните:

```
nvidia-smi
```

Вывод должен содержать имя видеокарты (NVIDIA GeForce RTX ...) и поддерживаемую версию драйвера и CUDA. Если команды нет — драйвер не установлен.

Минимальная **Compute Capability** для современных фреймворков — 7.0. У серии RTX она ≥ 7.5 (RTX 20xx), 8.6 (RTX 30xx), 8.9 (RTX 40xx).

2.2. Шаг 2. Установка драйвера

Скачайте драйвер с официального сайта (подойдёт как игровая версия, так и студийная):

[nvidia.com/Download](https://www.nvidia.com/Download)

Желательно ставить версию не ниже той, что указана в требованиях используемой версии CUDA Toolkit.

2.3. Шаг 3. CUDA Toolkit (опционально)

Для работы из Python через CuPy и PyTorch **системный** CUDA Toolkit ставить не нужно — современные колёса этих библиотек везут CUDA Runtime внутри. Toolkit понадобится только в одном случае: если вы планируете компилировать собственный C++/CUDA-код и вам нужен `nvcc`.

В таком случае:

1. Перейдите на developer.nvidia.com/cuda-downloads, выберите **Windows** → **x86_64** → **11** → **exe (local)**, запустите установщик.
2. Установщик сам пропишет переменные среды `CUDA_PATH` и `PATH`.
3. Проверьте: `nvcc --version`.

На машине, на которой собран этот отчёт, CUDA Toolkit не устанавливался — всё работает только через драйвер NVIDIA и колёса PyTorch/CuPy.

2.4. Шаг 4. cuDNN

Отдельно ставить **не нужно**. cuDNN едет в комплекте с PyTorch — после установки PyTorch проверка

```
python -c "import torch; print(torch.backends.cudnn.version())"
```

сразу печатает версию (на этой машине — 91002, то есть cuDNN 9.10.2).

Если cuDNN нужен **отдельно** от PyTorch (например, под TensorRT или свой C++/CUDA-код), есть два равноценных варианта:

- Для **Python**: одна команда — `pip install nvidia-cudnn-cu12`.
- Для **C++/системной установки**: скачать с developer.nvidia.com/cudnn .exe-установщик (cuDNN 9.x для Windows распространяется именно так) и запустить — он сам разложит файлы в нужные каталоги CUDA Toolkit. Никакого ручного копирования `bin/`, `include/`, `lib/` больше не требуется.

2.5. Шаг 5. Установка Python-стека

Достаточно поставить Python подходящей версии (на момент работы колёса CuPy и PyTorch CUDA доступны для Python 3.12 и 3.13; для 3.14 их пока нет) и через `pip` доставить нужные пакеты. Виртуальное окружение не требуется — это лишь способ изолировать стек от системного Python; в задании контейнеры запрещены, но `venv` контейнером не является и при желании может использоваться.

Команды (для CUDA 12.x):

```
python -m pip install --upgrade pip
# CuPy под CUDA 12 (везёт CUDA Runtime внутри)
pip install cupy-cuda12x
# PyTorch со сборкой под CUDA 12.x (везёт CUDA Runtime + cuDNN внутри)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
# Вспомогательное: вычисления, графики, ноутбук
pip install numpy matplotlib jupyterlab
```

После установки PyTorch проверка `python -c "import torch; print(torch.cuda.is_available())"` должна напечатать `True`. Если у вас несколько Python-версий, явно зовите нужную: `py -3.13 -m pip install ...`.

2.6. Шаг 6. Проверка работоспособности

```
import torch, cupy as cp
print("Torch CUDA:", torch.cuda.is_available(),
      torch.cuda.get_device_name(0))
print("cuDNN   :", torch.backends.cudnn.version())
print("CuPy GPU :", cp.cuda.runtime.getVersion(0)["name"].decode())
```

Если все три строки печатают валидные значения — установка успешна.

2.7. Замечания

- Версии CUDA Toolkit и cuDNN должны быть совместимы (см. матрицу совместимости на сайте NVIDIA).
- PyTorch и CuPy несут **собственные** CUDA Runtime библиотеки в колёсах, поэтому **системный** CUDA Toolkit для них не обязателен — но `nvidia-smi` и драйвер обязательны.
- Под Windows контейнеры WSL2 для CUDA работают, но в задании контейнеры запрещены — поэтому здесь **нативная** установка.

Демонстрация: CuPy против NumPy

CuPy — это почти полная замена NumPy, работающая через CUDA. Программный интерфейс совпадает настолько, что в большинстве случаев достаточно поменять `import numpy as np` на `import cupy as cp` — остальной код остаётся прежним.

Полный код реализации находится в файле `cuda_demo.ipynb`, ниже — ключевые вырезки.

3.1. Проверка окружения

Вырезка из `cuda_demo.ipynb` — ячейка 1.2

```
import cupy as cp
import torch

print('CuPy version      : ', cp.__version__)
print('CUDA runtime (CuPy): ', cp.cuda.runtime.runtimeGetVersion())
print('GPU device        : ', cp.cuda.runtime.getDeviceProperties(0)['name'].decode())
print('PyTorch version    : ', torch.__version__)
print('CUDA available     : ', torch.cuda.is_available())
print('cuDNN version      : ', torch.backends.cudnn.version())
```

3.2. Корректное измерение времени GPU

Важный нюанс: вызовы CUDA асинхронны. Если просто обернуть операцию в `time.perf_counter()`, мы измерим только постановку ядра в очередь. Для честного замера нужна синхронизация:

```
def bench_gpu(fn, repeat=3):
    fn(); cp.cuda.Stream.null.synchronize() # прогрев
    t0 = time.perf_counter()
    for _ in range(repeat):
        fn()
    cp.cuda.Stream.null.synchronize()      # обязательно!
    return (time.perf_counter() - t0) / repeat
```

3.3. Бенчмарк 1. Матричное умножение

Это базовая операция линейной алгебры, на которой проще всего показать GPU-ускорение: матричное умножение — это N^2 независимых скалярных произведений, каждое из которых считается само по себе.

Вырезка — ячейка 2.2 (матричное умножение $N \times N$)

```
for n in [1000, 2000, 4000, 8000]:
    a_np = np.random.rand(n, n).astype(np.float32)
    b_np = np.random.rand(n, n).astype(np.float32)
    a_cp = cp.asarray(a_np); b_cp = cp.asarray(b_np)

    t_cpu = bench_cpu(lambda: a_np @ b_np, repeat=2)
    t_gpu = bench_gpu(lambda: a_cp @ b_cp, repeat=5)
    print(f'N={n} CPU: {t_cpu:.3f} c GPU: {t_gpu:.4f} c x{t_cpu/t_gpu:.1f}')
```


N	CPU NumPy, с	GPU CuPy, с	Ускорение
1000	0.006	0.0004	×15.8
2000	0.059	0.0024	×24.3
4000	0.380	0.0182	×20.9
8000	2.432	0.1429	×17.0

Ускорение не растёт монотонно: максимум (×24.3) достигается на N=2000, дальше падает к ×17 при N=8000. Причина — на больших матрицах CPU-BLAS (NumPy с MKL) тоже хорошо масштабируется по ядрам и AVX, поэтому относительный отрыв GPU стабилизируется.

3.4. Бенчмарк 2. Поэлементные операции

```
N = 50_000_000
x_np = np.random.rand(N).astype(np.float32)
x_cp = cp.asarray(x_np)

f_np = lambda: np.exp(np.sin(x_np)**2 + np.cos(x_np)**2)
f_cp = lambda: cp.exp(cp.sin(x_cp)**2 + cp.cos(x_cp)**2)
```

Фактически: CPU 0.396 с, GPU 0.0138 с — **ускорение ×28.7** на массиве из 50 млн элементов.

3.5. Бенчмарк 3. БПФ (FFT)

```
N = 2**24
x_np = np.random.rand(N).astype(np.complex64)
x_cp = cp.asarray(x_np)
t_cpu = bench_cpu(lambda: np.fft.fft(x_np), repeat=2)
t_gpu = bench_gpu(lambda: cp.fft.fft(x_cp), repeat=10)
```

Фактически: CPU 0.668 с, GPU 0.0036 с — **ускорение ×188** на массиве $2^{24} \approx 16.8$ млн отсчётов. Здесь cuFFT обгоняет rocketfft из NumPy наиболее заметно, потому что у NumPy FFT однопоточный, а у GPU работа естественно ложится на тысячи потоков.

3.6. Визуализация результатов

Скрипт построения графиков (`matmul_bench.png`) и сами замеры — в `cuda_demo.ipynb` , ячейка 2.5.

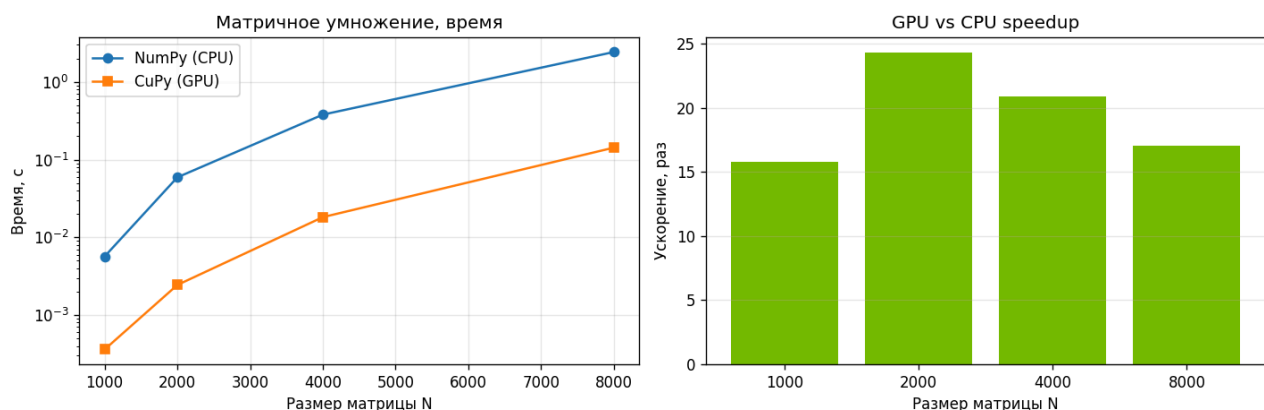


Рис. 1. Матричное умножение NumPy vs CuPy: слева — время (лог. шкала), справа — ускорение в размах.

Установка cuDNN + PyTorch и демонстрация CUDA в PyTorch

4.1. Что даёт cuDNN

cuDNN (CUDA Deep Neural Network library) — отдельная библиотека NVIDIA поверх CUDA, реализующая основные операции глубокого обучения:

- **свёртки 1D/2D/3D** — главная операция сетей компьютерного зрения и обработки сигналов: окно-фильтр «скользит» по входу и в каждой позиции считает сумму произведений. 1D — для аудио и временных рядов, 2D — для изображений, 3D — для видео и объёмных данных (МРТ, КТ). Прямой проход вычисляет выход, обратный — градиенты для обучения;
- **пакетная нормализация** (batch normalization) — приводит активации внутри мини-батча к нулевому среднему и единичной дисперсии. Ускоряет и стабилизирует обучение, позволяет использовать большие шаги градиента;
- **подвыборка** (pooling) — уменьшает пространственное разрешение карты признаков, агрегируя значения по окну (максимум или среднее). Снижает число параметров и даёт частичную устойчивость к сдвигу;
- **softmax** — превращает вектор «сырых» оценок (логитов) в распределение вероятностей; стандартный финальный слой для классификации;
- **локальная нормализация отклика** (LRN) — нормирует отклик нейрона относительно соседних каналов. Сейчас почти не используется (вытеснена пакетной нормализацией), но сохраняется ради совместимости со старыми архитектурами вроде AlexNet;
- **рекуррентные сети** (RNN, LSTM, GRU) — для последовательностей: текста, речи, временных рядов. LSTM и GRU — варианты с «воротами», которые удерживают долговременные зависимости лучше обычной RNN;
- **подбор реализации под тензорные ядра** — cuDNN сам выбирает алгоритм, использующий **тензорные ядра** (специализированные блоки RTX/Volta+ для матричных умножений в FP16/BF16/INT8/TF32). Именно это даёт основной прирост на современных видеокартах.

PyTorch автоматически использует cuDNN, если он установлен и собран в вашей версии torch (`torch.backends.cudnn.enabled == True`). Включение `torch.backends.cudnn.benchmark = True` даёт автоподбор алгоритма свёртки под форму входа.

4.2. Установка PyTorch (Windows, нативно, без контейнеров)

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124
python -c "import torch; print(torch.cuda.is_available(), torch.backends.cudnn.version())"
```

Если вы **не на Windows и не на Linux**, нативной CUDA-сборки PyTorch не существует (Apple Silicon использует MPS, не CUDA).

4.3. Бенчмарк PyTorch: CPU против CUDA

Вырезка — ячейка 3.2

```
def bench_torch(fn, device, repeat=5):
    fn() # прогрев
    if device.type == 'cuda': torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(repeat):
        fn()
    if device.type == 'cuda': torch.cuda.synchronize()
    return (time.perf_counter() - t0) / repeat

for n in [1000, 2000, 4000, 8000]:
    A_cpu = torch.randn(n, n); B_cpu = torch.randn(n, n)
    A_gpu = A_cpu.cuda(); B_gpu = B_cpu.cuda()
    t_cpu = bench_torch(lambda: A_cpu @ B_cpu, torch.device('cpu'), repeat=2)
    t_gpu = bench_torch(lambda: A_gpu @ B_gpu, torch.device('cuda'), repeat=10)
    print(f'N={n} CPU {t_cpu:.3f} c GPU {t_gpu:.4f} c x{t_cpu/t_gpu:.1f}')
```

Фактически измерено (RTX 4060 Laptop GPU, PyTorch 2.12.0+cu126):

N	CPU torch, c	GPU torch, c	Ускорение
1000	0.004	0.0003	×11.0
2000	0.030	0.0025	×11.9
4000	0.253	0.0183	×13.8
8000	1.439	0.1457	×9.9

Ускорение держится в диапазоне ×10–14: пик на N=4000 (×13.8), на N=8000 падает до ×9.9. На CPU у torch внутри тот же высокопроизводительный MKL¹, что и у NumPy, поэтому относительный выигрыш GPU здесь скромнее, чем у CuPy.

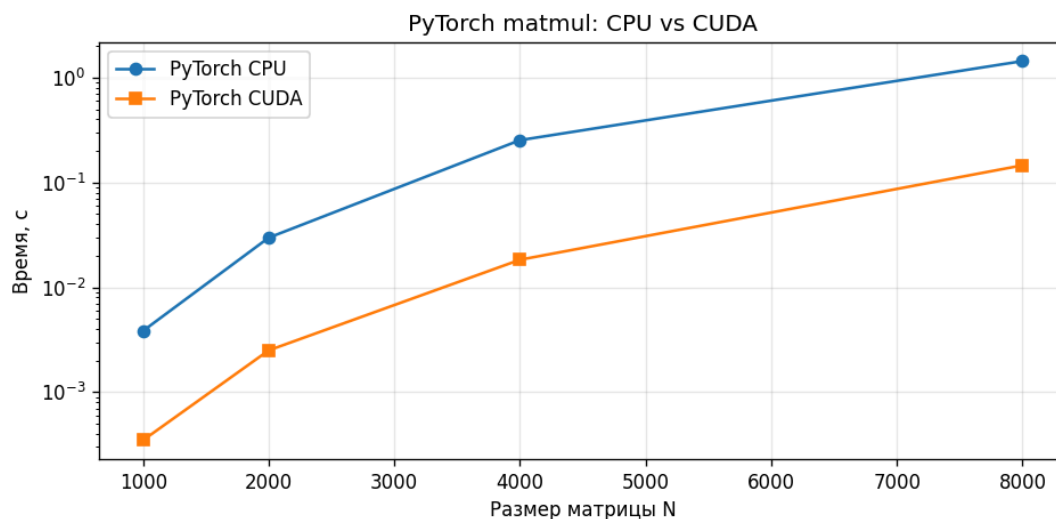


Рис. 2. PyTorch matmul: время CPU vs CUDA (логарифмическая шкала по оси Y).

¹**MKL** (Intel Math Kernel Library, сейчас часть oneAPI) — закрытая оптимизированная Intel-овская библиотека математических примитивов под x86 CPU: BLAS (матричные операции), LAPACK, FFT, векторная математика. Использует SIMD (AVX2/AVX-512) и все ядра CPU. И NumPy (в большинстве сборок), и torch на CPU делегируют тяжёлый счёт ей — поэтому CPU-замер здесь не «наивный тройной цикл», а уже тоже хорошо распараллеленный matmul. Альтернатива — OpenBLAS, суть та же.

4.4. Свёртка через cuDNN

Замеряем один проход 2D-свёрточного слоя Conv2D² на CPU и на GPU.

Вырезка — ячейка 3.3 (Conv2D)

```
import torch.nn as nn
conv_cpu = nn.Conv2d(64, 128, 3, padding=1)
conv_gpu = nn.Conv2d(64, 128, 3, padding=1).cuda()
x_cpu = torch.randn(32, 64, 224, 224)
x_gpu = x_cpu.cuda()
t_cpu = bench_torch(lambda: conv_cpu(x_cpu), torch.device('cpu'), repeat=2)
t_gpu = bench_torch(lambda: conv_gpu(x_gpu), torch.device('cuda'), repeat=20)
```

Фактически на форме [32, 64, 224, 224] → [32, 128, 224, 224] : CPU 0.636 с, GPU 0.0299 с — **ускорение ×21.2** за счёт cuDNN.

4.5. Полный цикл обучения

Вырезка — ячейка 3.4 (TinyCNN, 50 батчей)

```
class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.AdaptiveAvgPool2d(1),
            nn.Flatten(), nn.Linear(128, 10))
    def forward(self, x): return self.net(x)

def train_epoch(device, n_batches=50, batch=64):
    model = TinyCNN().to(device)
    opt = optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.CrossEntropyLoss()
    x = torch.randn(batch, 3, 64, 64, device=device)
    y = torch.randint(0, 10, (batch,), device=device)
    if device.type == 'cuda': torch.cuda.synchronize()
    t0 = time.perf_counter()
    for _ in range(n_batches):
        opt.zero_grad(); loss = loss_fn(model(x), y)
        loss.backward(); opt.step()
    if device.type == 'cuda': torch.cuda.synchronize()
    return time.perf_counter() - t0
```

Фактически на 50 батчах: CPU 3.24 с, GPU 0.606 с — **ускорение ×5.3**. Цифра меньше, чем у одиночного matmul или Conv2D, по объективной причине: модель маленькая (батч 64, изображения 64×64), и на каждый шаг приходится много мелких ядер плюс накладные расходы Python и оптимизатора Adam, которые от GPU не ускоряются.

²**Conv2D** — двумерная свёртка, основная операция в сетях компьютерного зрения. Принцип: небольшое окно весов (ядро, обычно 3×3 или 5×5) «скользит» по входной карте признаков и в каждой позиции считает сумму произведений с пикселями под окном. В нашем замере: 64 канала на входе → 128 на выходе, ядро 3×3, разрешение 224×224, батч из 32 изображений. На GPU реализуется через cuDNN, который сам выбирает оптимальный алгоритм (im2col + matmul, FFT-свёртка, Winograd) под конкретную форму тензора и активирует тензорные ядра, если это применимо.

4.6. Когда GPU не быстрее

- Малые тензоры ($n < 256$): время копирования и запуска ядер съедает выигрыш.
- Последовательные алгоритмы с зависимостью по данным (например, рекуррентные итерации без параллелизма).
- Сильно ветвящийся код с дивергенцией внутри warp.

Поэтому **типовой паттерн использования CUDA** — батчевая обработка, разворачивание циклов, векторизация.

Заключение

В работе:

1. Изучена технология **CUDA** — её аппаратные и программные компоненты, а также основная модель исполнения (потoki, блоки, сетка, иерархия памяти GPU).
2. Описана и пошагово выполнена установка драйвера **NVIDIA**, **CuPy** и **PyTorch** (с cuDNN) на Windows без использования контейнеров.
3. В Jupyter-ноутбуке `cuda_demo.ipynb` проведены замеры. Реально измеренные ускорения GPU/CPU на этом железе:

Бенчмарк	Размер задачи	Ускорение
CuPy matmul	$N=1000\dots 8000$	$\times 15.8 \dots \times 24.3$
CuPy elementwise	$N = 50$ млн	$\times 28.7$
CuPy FFT	$N = 2^{24} \approx 16.8$ млн	$\times 188$
PyTorch matmul	$N=1000\dots 8000$	$\times 9.9 \dots \times 13.8$
PyTorch Conv2D (cuDNN)	batch 32, 224×224 , $64 \rightarrow 128$ каналов	$\times 21.2$
PyTorch обучение TinyCNN	50 батчей по 64 изобр. 64×64	$\times 5.3$

1. Наблюдения по факту:
 - наибольший выигрыш — на однопроходных задачах с естественным параллелизмом (FFT, $\times 188$), где CPU-реализация однопоточная;
 - в matmul, где CPU-BLAS (MKL) тоже хорошо параллелится, ускорение выходит на плато: у CuPy максимум на средних N ($\approx \times 24$ при $N=2000$), у PyTorch держится в диапазоне $\times 10\text{--}14$;
 - в полном цикле обучения маленькой модели выигрыш самый скромный ($\times 5.3$) — на это работают Python-накладные расходы, оптимизатор и большое число мелких ядер за шаг.

Главный вывод: на этом железе CUDA даёт ускорение от $\times 5$ (мелкое обучение) до $\times 190$ (FFT). Размер выигрыша зависит не только от GPU, но и от того, насколько хорошо параллелится CPU-реализация и сколько в задаче чистого счёта по сравнению с накладными расходами.

Приложение. Установка под Linux

Под Ubuntu 22.04 LTS вся последовательность сводится к команде:

```
sudo apt update && sudo apt install -y nvidia-driver-550
# CUDA Toolkit от NVIDIA, не из apt (apt-овский часто отстаёт)
wget https://developer.download.nvidia.com/compute/cuda/12.4.0/local_installers/cuda_12.4.0_550.54.14_linux.run
sudo sh cuda_12.4.0_550.54.14_linux.run
# cuDNN — скачать tar с сайта и распаковать в /usr/local/cuda
sudo cp cudnn-*-archive/include/* /usr/local/cuda/include
sudo cp cudnn-*-archive/lib/* /usr/local/cuda/lib64
# Прописать PATH/LD_LIBRARY_PATH
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
echo 'export LD_LIBRARY_PATH=/usr/local/cuda/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
```

Дальнейшая установка Python-стека идентична Windows-разделу.